CHAPTER 33

Tool Descriptions, Naming and Affordances

In this chapter

Introduction: Thirty-Four Coin Flips . . . 3

33.1 Core Concepts . . . 4

33.2 The Claim, and What It Rests On . . . 5

33.3 What Goes In a Description . . . 6

33.4 Names . . . 6

33.5 Parameters, Enums and the Rest of the Surface . . . 7

33.6 What a Tool Returns Is Also an Affordance . . . 7

33.7 Measuring a Catalogue, Honestly . . . 8

33.8 The Statistics You Need to Say Anything . . . 8

33.9 Architecture: Where the Words Live . . . 9

33.10 Figures . . . 9

33.11 Examples . . . 10

33.16 Code Walkthrough: A Scorer That Admits What It Is . . . 11

33.17 Mini Project: A Catalogue Linter and a Confusability Harness . . . 12

33.18 Hands-on Labs . . . 15

33.19 Lab Solutions . . . 15

33.20 Best Practices . . . 16

33.21 Common Mistakes . . . 16

33.22 Performance Considerations . . . 17

33.23 Security Considerations . . . 18

33.24 Real Production Examples . . . 18

33.25 Certification Notes . . . 19

33.26 Twenty Practice Questions . . . 19

33.27 Answers . . . 20

33.28 Ten Practical Exercises . . . 21

33.29 Five Challenge Labs . . . 21

33.30 Ten Interview Questions . . . 21

33.31 Summary . . . 22

33.32 Glossary . . . 23

33.33 Further Reading 24

Part 6 of 13 · Chapter 33 of 64. Next: Chapter 34 — Error Contracts, Validation and Idempotency.

NOTE

Learning Objectives

By the end of this chapter you will be able to:

1. Explain why the description is *"by far the most important factor in tool performance"*, and what that claim rests on.
2. Write a description covering the five things the documentation asks for, including the two everybody omits.
3. Name tools so that selection is unambiguous as the library grows, and say why namespacing matters most under tool search.
4. Diagnose **selection ambiguity** between two tools, and fix it with a sentence rather than with a schema change.
5. Shape a tool's **response** so it returns high-signal, semantically stable information and nothing else.
6. Measure a catalogue's confusability offline, and say honestly what such a measurement does and does not establish.
7. Apply Chapter 24's discipline to a description change: pair the comparison, report an interval, and publish a negative result when one is what you have.
8. Justify a long description economically, using Chapter 30's cache arithmetic.

NOTE

Prerequisites

Chapter 8 for Wilson intervals and why a proportion without one is not a finding. **Chapter 20** for the prompting rules that apply unchanged to a description — be clear and direct, give the reason with the rule, say what to do rather than what not to do, and make examples relevant, diverse and structured. **Chapter 22** for structure as a learned convention and for the practice of publishing a detector's blind spot. **Chapter 24** for pairing, power and the winner's curse. **Chapter 30** for the cached prefix that makes a long description nearly free. **Chapter 31** for the fact that Claude sees only the schema and the result. **Chapter 32** for the fields, the name regex, the budget and the consolidation arithmetic.

Deferred by name: **error contracts, call-site validation and idempotency** are **Chapter 34**; **orchestration across many tools and turns** is **Chapter 35**; **tools whose descriptions you did not write** are **Part 7 (Chapters 36–40)**; **retrieval over a large tool library** connects to **Chapter 53**.

Introduction: Thirty-Four Coin Flips

A support catalogue has five tools: order status, shipment status, refund, knowledge-base search, ticket search. Each has a name and a one-line description. `order_status` says *"Get the status of an order."* `shipment_status` says *"Get the status of a shipment."*

§33.17 puts forty realistic customer requests through that catalogue using a deliberately simple, transparent scorer, and finds something more uncomfortable than a low score. **Thirty-four of the forty requests end in a tie.** Not a wrong answer — an *absent* one. The top two tools score identically, and which one "wins" is settled by an arbitrary rule that has nothing to do with the request. In the harness that rule is alphabetical order. In production it is whatever the model happens to fall back on.

That is the honest shape of a bad tool catalogue. It does not usually pick the wrong tool for a reason. It fails to distinguish, and then something else decides.

Rewrite the same five tools — same names' worth of tools, same schemas, same count, only the words changed to follow the documented guidance — and ties fall from **34 to 16**, the mean margin between first and second place rises from **0.208 to 1.011**, and proxy accuracy goes from **12/40 to 22/40**.

Then the chapter has to be careful, and this is the more important half. Those two proportions carry 95 per cent Wilson intervals of **[0.181, 0.454]** and **[0.398, 0.693]**, and **the intervals overlap**. On the unpaired comparison, at forty cases, a twenty-five-point improvement is **not established**. Chapter 24 gave the remedy and it works here exactly as advertised: pair the trials, look only at the fourteen cases where the two catalogues disagree — twelve improvements against two regressions — and an exact test returns **p = 0.0129**. The difference is real; the unpaired reading simply could not see it.

So this chapter has two subjects that turn out to be the same subject. The first is what to write in a tool description, which the documentation states plainly and which almost nobody does. The second is how you would ever know whether it helped — because *"write better descriptions"* is exactly the kind of advice that is obviously true, universally agreed, and almost never measured. §33.17 also publishes a change that sounds like an improvement, produces **zero discordant pairs**, and is therefore not one.

One caveat is owed up front and repeated throughout, because the chapter is worthless without it. **The harness does not measure Claude.** It runs with no key, like every mini project in this book. What it measures is a property of the *catalogue*: how far apart two tools are in the only material Claude is given to tell them apart. Chapter 31 quoted the sentence this rests on — *"Claude never sees your implementation; it only sees the schema you provided and the result you returned."* The input to tool selection is a finite, inspectable string. A catalogue whose tools a bag-of-words scorer cannot separate is one whose names and descriptions are carrying no signal at all, and no amount of model capability is being *helped* by them.

33.1 Core Concepts

DEFINITION

Affordance

What an interface communicates about its own use. A tool's affordances are its name, its description, its parameter names and descriptions, its enum values, and the shape of what it returns — everything the model can see.

DEFINITION

Selection ambiguity

The condition in which two or more tools are plausible answers to the same request. The documented remedy is *"fewer, more capable tools"*, and the cheaper remedy is a sentence saying when **not** to use each.

DEFINITION

Margin

The gap between the best-scoring tool and the runner-up. A high accuracy with a low margin is a catalogue that is being right by luck.

DEFINITION

Tie

A trial where the top two tools score identically. Not a wrong answer; an absent one, resolved by something arbitrary.

DEFINITION

Response shaping

Designing what a tool returns: *"only high-signal information"*, *"semantic, stable identifiers (for example, slugs or UUIDs) rather than opaque internal references"*, and *"only the fields Claude needs to reason about its next step."*

DEFINITION

Namespacing

Prefixing a tool name with its service — `github_list_prs`, `slack_send_message` — so that *"tool selection is unambiguous as your library grows"*.

33.2 The Claim, and What It Rests On

The documentation does not hedge: *"**Provide extremely detailed descriptions.** This is by far the most important factor in tool performance."*

That is a strong claim about a soft-looking field, and it is worth understanding why it should be true rather than accepting it. Three reasons, all established earlier in this book.

The description is the entire interface. Chapter 31 recorded that Claude never sees your implementation. A function signature in a normal codebase is backed by source someone can read; a tool schema is not. Every fact about behaviour that is not in the description does not exist.

The description is a prompt. It is assembled into the tool-use system prompt Chapter 32 quoted, which sits above your own system prompt in the constructed prefix. Everything Chapter 20 established about prompting applies to it without modification: be clear and direct, **give the reason with the rule**, say what to do rather than only what not to do, and remember that Claude *"is smart enough to generalize from the explanation."*

The description is nearly free. It lives in the `tools` array, which Chapter 30 established is the first level of the cached prefix. After the first request it is read at **0.1×**. §33.17 prices the trade: the extra 1,500 tokens that turn five one-liners into five proper descriptions cost **150 base-input units per cached request**, while one wrong tool call on a 20,000-token cached history costs **2,000 units of input** plus a

full extra generation — **13.3 times** as much, before counting the latency or the side effect.

That third point deserves its own line, because it inverts an instinct. Brevity in a tool description is not a saving. It is a bet that the model will infer what you did not say, priced at roughly thirteen to one against you.

33.3 What Goes In a Description

The documented list has five items, and the last two are the ones almost every real catalogue omits. A description should explain:

- **What the tool does.**
- **When it should be used — and when it shouldn't.**
- **What each parameter means and how it affects the tool's behaviour.**
- **Any important caveats or limitations**, *"such as what information the tool does not return if the tool name is unclear."*
- With a floor: *"Aim for at least 3–4 sentences for each tool description, more if the tool is complex."*

The two neglected ones are **when not to use it** and **what it does not return**, and they are neglected because they feel like documentation for a reader who already chose the tool. They are not. They are the only two items that do work at *selection* time, which is the moment the model is deciding between your tools rather than reading about one.

Consider the pair from §33.17. *"Get the status of an order."* and *"Get the status of a shipment."* Both are true. Both are clear. Neither helps at all with *"where is my parcel right now"*, because nothing in either sentence says which one owns parcels. Add one clause to each —

> *"Do not use it to find out where a parcel is or when it will arrive — that is* `commerce_shipment_tracking`*."*
> > *"Do not use it for payment, invoice or cancellation questions — that is* `commerce_order_status`*."*

— and the pair is separated, in both directions, by naming the alternative. Chapter 20's rule about saying what to do rather than what not to do is not violated here, because the sentence does both: it rules the tool out **and** routes to the right one. A bare prohibition would be worse than nothing.

> TIP
>
> **The reciprocal-exclusion rule**
>
> Whenever two tools could answer the same request, each description should name the other and say which kinds of question belong to it. One sentence, written twice, in opposite directions. It is the cheapest fix in this chapter and §33.17 measures it as the bulk of the improvement.

33.4 Names

Chapter 32 gave the constraint — `^[a-zA-Z0-9_-]{1,64}$` — and the guidance: *"When your tools span multiple services or resources, prefix names with the service (for example,* `github_list_prs`*,* `slack_send_message`*). This makes tool selection unambiguous as your library grows, and is especially important when using tool search."*

Three things a name is doing, none of which is identity.

It is the shortest description. It is read on every selection, before any description is weighed. A name that says `refund` says less than `commerce_issue_refund`, and the latter costs three tokens.

It carves the space. Namespacing groups tools into families, and a family is something the model can rule in or out wholesale. Sixty-four characters is a generous budget; there is no reason to economise.

It is the retrieval key under tool search. Chapter 30 introduced `defer_loading`: a deferred tool is not in the prefix, and *"when the model discovers a deferred tool through tool search, the definition is appended inline as a* `tool_reference` *block."* Under that regime, the model is searching a library rather than reading a list, and a name that does not say which service it belongs to is a document with no title. This is why the documentation flags namespacing as *"especially important when using tool search"*, and it is the mechanism behind the advice rather than a style preference.

33.5 Parameters, Enums and the Rest of the Surface

The description field is not the only prose the model sees. §33.17's scorer reads the whole surface, and so does Claude: **tool name, description, parameter names, parameter descriptions and enum values**.

That has practical consequences. A parameter called `q` communicates nothing; `query` communicates a little; `query` with *"Natural-language search phrase"* communicates what kind of thing to put in it. An enum whose members are `1`, `2`, `3` throws away the best affordance in JSON Schema — Chapter 32 showed that `enum` is one of the few constraints the grammar actually enforces, and its members are also readable text.

Chapter 32's consolidation argument returns here in a different key. Collapsing nine pull-request tools into one `github_pr` tool with an `action` enum was justified there by the request-level budget; the documented justification is about selection: *"Fewer, more capable tools reduce selection ambiguity and make your tool surface easier for Claude to navigate."* The `action` enum turns a choice between nine tools into a choice between eight enum members **inside one tool that has already been selected**, which is a strictly easier problem — and the enum members are themselves affordances.

Chapter 32 recorded that `input_examples` are schema-validated and cost 20–200 tokens each, and the documented priority: *"Prioritize descriptions, but consider using* `input_examples` *for complex tools."* Chapter 20's rules govern them — relevant, diverse, structured, three to five — and the diversity that matters is **showing an optional parameter used and omitted**, which is the thing a schema cannot express.

33.6 What a Tool Returns Is Also an Affordance

The documented guidance is easy to skim past and is one of the highest-leverage items in this chapter: *"Design tool responses to return only high-signal information. Return semantic, stable identifiers (for example, slugs or UUIDs) rather than opaque internal references, and include only the fields Claude needs to reason about its next step. Bloated responses waste context and make it harder for Claude to extract what matters."*

Three separate instructions.

Semantic and stable identifiers. An internal row id of `88214` is opaque and, worse, unstable across environments. A slug like `order-2026-4471` is readable, is the same thing in staging and production, and can be reasoned about — the model can tell two of them apart without another tool call.

Only the fields needed for the next step. A result is not an API response you happen to have; it is a message to a component with a fixed attention budget. Chapter 5 established that attention is quadratic and its budget fixed, which is why retrieval beats context-stuffing; a tool that returns forty fields when three are needed is context-stuffing with extra steps.

Bloat costs twice. Once in tokens, and once in Chapter 31's block accounting, where every result is a block and blocks are what the 20-block cache lookback window counts.

There is a fourth consideration this book has already raised and will raise again in Chapter 58: whatever the tool returns is **untrusted input**, and the smaller and more structured it is, the less room there is for anything to hide in it.

33.7 Measuring a Catalogue, Honestly

Here is the methodological problem, stated plainly. "Better descriptions improve tool selection" is a claim about a model's behaviour. Measuring it properly requires running requests against the model with an evaluation set, which Chapter 24 costed at real money and real design effort. This chapter's project runs offline with no key.

So the project does not measure that. It measures something narrower and genuinely checkable: **how distinguishable the tools in a catalogue are, in the material the model is given.** The scorer is an idf-weighted bag-of-words affinity — transparent, deterministic, and far weaker than a language model. Chapter 5's lexical retrieval mis-ranking is the direct ancestor, and the same caveat applies: a weak lexical method is a **lower bound**.

What follows from a lower bound, and what does not:

- A catalogue the scorer **cannot** separate is one where the names and descriptions carry no distinguishing signal. That is a fact about your strings, not a guess about the model.
- A catalogue the scorer **can** separate is not thereby proven good. A stronger reader may still confuse tools that are lexically distinct but semantically overlapping.
- The **tie count** is the most trustworthy number the harness produces, because a tie is a property of the scoring function meeting the text, not an opinion about which tool is better.

Everything in §33.17 is reported with that boundary drawn. The chapter would rather publish a narrow claim that holds than a broad one it cannot support — the same discipline Chapter 12 applied when a hypothesis failed its own confidence intervals.

33.8 The Statistics You Need to Say Anything

Chapter 24 established the tools; this chapter is a small, complete worked example of why they are not optional.

A proportion without an interval is not a finding. 12/40 and 22/40 look decisive. Their Wilson intervals are [0.181, 0.454] and [0.398, 0.693], and they overlap. At forty cases the unpaired comparison establishes nothing.

Always pair. The two catalogues answered the *same* forty requests, so the trials can be matched. Of forty, twenty-six agreed — ten right on both, sixteen wrong on both — and carry no information about the difference. All the evidence is in the fourteen **discordant** pairs, split twelve improvements to two regressions, which an exact two-sided test puts at **p = 0.0129**. Chapter 24 measured pairing as worth 3.51× in sample size; here it is the difference between a finding and a shrug.

Compute the size you need before you believe a number. Reaching a ±0.05 interval around 0.55 needs **410** cases. Forty is fine for detecting a large paired difference and hopeless for estimating a level.

Expect the winner's curse. Chapter 24 measured twenty identical prompts whose best performer fell from 83.82 per cent to 79.96 per cent on fresh data. If you try eight description variants and keep the best, you have not found the best description; you have found the luckiest one. Confirm on held-out requests.

33.9 Architecture: Where the Words Live

Descriptions belong in the **cached prefix**, which means three things operationally.

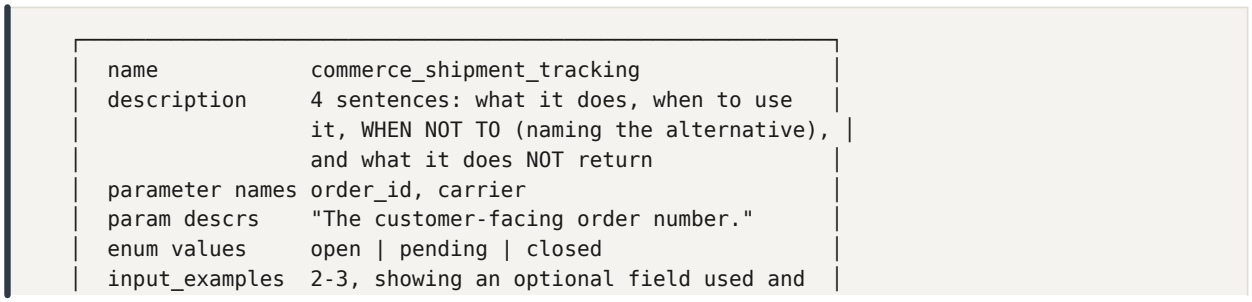
They are **written once and read constantly**, at 0.1× after the first request — the economic argument of §33.2.

They are **byte-sensitive**. Chapter 32 measured an unstable `tools` array at 3.45× the input line. A description assembled with a version string, a timestamp or a per-user detail is the most expensive kind of prose you can write.

They are **versionable artefacts, not comments**. A description change is a behaviour change with no type signature and no compiler. It belongs in review, in a changelog, and behind a regression suite — Chapter 24's pinned artefact rule applies: cases, grader, rule and baseline, and changing any of them is a breaking change.

One asymmetry from Chapter 23 is worth restating because it is genuinely useful here. The **grammar cache** is invalidated by schema *structure* and by the tool set, **not** by `name` or `description` changes. The **prompt cache** is invalidated by any byte. So editing a description is free for one cache and expensive for the other, and the two are keyed differently on purpose.

33.10 Figures



```
   │                  omitted                                  │
   └──────────────────────────────────────────────────────────┘

                    ▲
                    │  this is the WHOLE interface
                    │  "Claude never sees your implementation"
                    │
   your code  ──────┘   (invisible)
   What the model does NOT have: your source, your tests, your
   naming conventions, last week's incident, or the reason the
   two tools exist separately at all.
```

```
  BEFORE   five one-line descriptions
     40 requests
     ├─ 34 TIES              top two tools score identically
     │     └─ resolved by an arbitrary rule (here: alphabetical)
     ├─  6 decided on signal
     └─ accuracy 12/40 = 0.3000   95% Wilson [0.1807, 0.4543]
  AFTER    same tools, same schemas, documented descriptions
     40 requests
     ├─ 16 TIES
     ├─ 24 decided on signal
     └─ accuracy 22/40 = 0.5500   95% Wilson [0.3983, 0.6929]
  UNPAIRED   the intervals OVERLAP -> nothing established
  PAIRED     26 concordant (no information)
             14 discordant: 12 improved, 2 regressed
             exact two-sided test  p = 0.0129  -> established
  mean margin (first minus second): 0.208 -> 1.011
```

33.11 Examples

Example 1 — the one-liner that decides nothing.

```
{"name": "order_status",
 "description": "Get the status of an order."}
```

Example 2 — the same tool, following the documented list.

```
{"name": "commerce_order_status",
 "description":
     "Look up the commercial state of a customer order: whether it "
     "is placed, paid, cancelled or fulfilled, and what was bought. "
     "Use this for questions about payment, invoices, order contents "
     "or cancellation. Do not use it to find out where a parcel is "
     "or when it will arrive - that is commerce_shipment_tracking. "
     "It does not return carrier tracking events or delivery "
     "estimates."}
```

Example 3 — the reciprocal half. The pair only works written twice:

```
"Do not use it for payment, invoice or cancellation questions - "
"that is commerce_order_status. It does not return prices, order "
"contents or refund state."
```

Example 4 — parameters carry description too.

```
"properties": {
    "query": {"type": "string",
              "description": "Natural-language search phrase."},
    "status": {"type": "string", "enum": ["open", "pending", "closed"],
               "description": "Restrict to open, pending or closed."}}
```

Example 5 — a response shaped for the next step.

```
return {"order": "order-2026-4471",        # semantic, stable slug
        "state": "paid",                    # not 40 fields
        "refundable_minor": 4200}           # what the model needs next
```

Example 6 — the measurement that keeps you honest.

```
before = run(requests, OLD_CATALOGUE)
after = run(requests, NEW_CATALOGUE)      # same requests: PAIRED
p = paired(before, after)                 # 12 improved, 2 regressed
mcnemar_exact(p["improved"], p["regressed"])   # 0.0129
```

33.16 Code Walkthrough: A Scorer That Admits What It Is

`affordance.py` opens with a warning rather than an import, and the warning is load-bearing:

```
"""affordance.py - measuring a CATALOGUE, not a model.
...What it CAN measure is a property of the catalogue itself - how far
apart two tools are, in the only material Claude is given to tell them
apart... Treat it as a LOWER BOUND on how distinguishable your tools
are. A catalogue this scorer can separate is not thereby proven good.
A catalogue it CANNOT separate is one where the names and
descriptions are carrying no signal at all.
"""
```

Every other mini project in this book measures something the API documents. This one measures a proxy, and the only defensible way to publish a proxy is to say so at the top of the file, in the report, and in every claim derived from it.

The function that decides what the model can see is four lines and is the chapter in miniature:

```
def surface(tool):
    """Everything Claude is given about a tool, as one string."""
    parts = [tool.name, tool.description]
    props = (tool.input_schema.get("properties") or {})
    for name, sub in props.items():
        parts.append(name)
        if isinstance(sub, dict) and sub.get("description"):
            parts.append(sub["description"])
        if isinstance(sub, dict) and sub.get("enum"):
            parts.extend(str(v) for v in sub["enum"])
    return " ".join(parts)
```

Name, description, parameter names, parameter descriptions, enum values. Nothing else. If a fact about your tool is not in that string, it is not available at selection time — which is Chapter 31's *"Claude never sees your implementation"* turned into code.

The weighting is inverse document frequency computed over **the catalogue's own text**, and that choice is deliberate:

```
    return {t: math.log((n + 1) / (c + 0.5)) for t, c in df.items()}
```

A word that appears in every tool's description separates nothing and is weighted down accordingly. This is precisely where a catalogue of near-identical one-liners is punished: *"status"* appears in two of five descriptions and *"the"* in all of them, so almost the entire vocabulary is worthless for discrimination and the scores collapse together.

Which produces the number the chapter trusts most:

```
def ties(trials):
    """Trials decided by nothing at all: the top two scored equal.
    A tie is not a wrong answer, it is an ABSENT answer that some
    arbitrary rule then resolves - here, alphabetical order.
    """
    return sum(1 for t in trials if t.margin == 0.0)
```

Publishing the tie count is what stops the confusion matrix being misread. Without it, §33.17's before-matrix looks as though `ticket_search` exerts some semantic pull over unrelated requests. It does not. It wins because the scores are equal and it sorts last. Reporting the confusion without the ties would be a true table telling a false story.

`lintdesc.py` turns the documented list into checks, and one rule is included specifically so the chapter can publish its failure — see Result 4.

33.17 Mini Project: A Catalogue Linter and a Confusability Harness

Folder structure.

```
ch33/
  affordance.py    the surface, the proxy scorer, ties, pairing
  lintdesc.py      the documented description rules, as checks
  catalogue33.py   five tools written twice, and 40 requests
  report.py        every published figure
  test_ch33.py     16 tests
  evalkit.py       copied from ch8/   (Wilson, sample size)
  tooldef.py       copied from ch32/  (Tool, the budget)
  enforce.py       copied from ch32/
  schema.py        copied from ch23/
  cacheplan.py     copied from ch30/
  costmodel.py     copied from ch11/
```

The catalogue is written twice. **BEFORE** is what tool definitions look like in most codebases: a name, one sentence, no parameter descriptions, and two tools whose jobs sound the same. **AFTER** changes only the words — same number of tools, same schemas, same parameters — and follows the documented guidance. Both are marked `BUILT`.

Result 1 — thirty-four of forty decided by nothing

```
before  12/40 = 0.3000   95% Wilson [0.1807, 0.4543]
after   22/40 = 0.5500   95% Wilson [0.3983, 0.6929]
difference +0.2500; the intervals OVERLAP
paired: improved=12 regressed=2 both=10 neither=16 discordant=14
McNemar exact on the 14 discordant pairs: p = 0.0129 -> established
trials decided by a TIE: before=34/40  after=16/40
```

Read the last line first. **Thirty-four of forty** requests against the one-line catalogue end with the top two tools scoring identically. The catalogue does not express a wrong preference; it expresses no preference, and something arbitrary finishes the job. After the rewrite, ties fall to sixteen — still high, and honestly so: five tools in one domain remain genuinely close, and the harness is a weak reader.

Then the statistics, which are the more transferable lesson. A jump from 30 to 55 per cent looks unarguable and the two Wilson intervals **overlap**, so on the unpaired comparison at n = 40 nothing has been established. Pairing rescues it exactly as Chapter 24 said it would: twenty-six of the forty trials agree and carry no information, and the fourteen that disagree split twelve to two, which an exact two-sided test puts at **p = 0.0129**.

Both readings are true at once, and a team that reported only the first would be overclaiming while a team that reported only the intervals would abandon a real improvement. This is the whole argument for pairing in nine lines of output.

Result 2 — the evidence is off the diagonal

```
BEFORE
  kb_search         -> kb_search=1, ticket_search=7
  order_status      -> shipment_status=3, ticket_search=5
  refund            -> refund=3, ticket_search=5
  shipment_status   -> shipment_status=1, ticket_search=7
  ticket_search     -> order_status=1, ticket_search=7
AFTER
  commerce_issue_refund -> commerce_issue_refund=5, ...=3
  commerce_order_status -> commerce_order_status=2,
                           commerce_shipment_tracking=3, ...=3
  commerce_shipment_tracking -> commerce_shipment_tracking=4,
                           commerce_order_status=2, ...=2
  support_article_search -> support_article_search=5,
                           support_ticket_search=3
  support_ticket_search  -> support_ticket_search=6, ...=2
margin (top score minus runner-up), mean:
  before  all=0.208  on the ones it got wrong=0.050 (n=28)
  after   all=1.011  on the ones it got wrong=0.559 (n=18)
```

The before-matrix must be read with Result 1's tie count in hand. `ticket_search` appears to swallow the catalogue; in fact it collects the ties, because the tie-break is alphabetical and it sorts last. **The correct summary is not "everything goes to ticket search" but "almost nothing is decided by the text at all."**

The after-matrix is more interesting because it still contains errors, and they are the *right* errors: `commerce_order_status` losing three requests to `commerce_shipment_tracking`, and `support_article_search` losing three to `support_ticket_search`. Those are the two genuinely adjacent pairs in the domain, and a weak lexical reader confusing them is expected. The nine-way scatter of the

before-matrix is not a harder version of the same problem; it is a different problem, and the mean margin says so — **0.208 to 1.011**, and on the trials it still gets wrong, **0.050 to 0.559**. Even the mistakes are now being made for a reason.

Result 3 — a change that is not an improvement

```
baseline  22/40 = 0.5500  [0.3983, 0.6929]
reworded  22/40 = 0.5500  [0.3983, 0.6929]
paired discordant pairs: 0
VERDICT: no evidence of a difference at n=40
n needed for a +/-0.05 interval at p=0.55: 410
```

Rewriting *"Use this"* as *"Use this tool"* across all five descriptions — the kind of edit that gets made in review and defended on grounds of clarity — produces **zero discordant pairs**. Not a small effect: no effect at all, on this harness, at this sample size. It is published because the chapter's own thesis is that words matter, and the honest form of that thesis is *some* words matter, and you have to measure which.

The last line is the one to carry into your own work. Estimating a *level* to within ±5 points needs **410** cases. Forty cases can detect a large paired difference and cannot tell you what your selection accuracy is.

Result 4 — the linter, and a blind spot it publishes

```
BEFORE  errors=10 warns=11
   [ERROR] order_status  TOO_SHORT: 1 sentence(s); the guide asks
                         for at least 3-4
   [ERROR] order_status  NO_WHEN_NOT: does not say when the tool
                         should NOT be used
   [WARN ] order_status  NO_OMISSIONS: does not say what the tool
                         does not return
   [WARN ] order_status  UNDOCUMENTED_PARAM: no description on:
                         include_history, order_id
   ... 15 more
AFTER   errors=0 warns=0
```

Ten errors and eleven warnings across five tools, every rule traceable to a sentence in the documentation. All five tools fail both error rules — too short, and silent about when not to use them.

Now the part worth more than the passing score. The linter has an **OVERLAP** rule that flags two descriptions sharing more than half their vocabulary, on the documented grounds that selection ambiguity is the reason to consolidate. On the BEFORE catalogue that rule **does not fire** — and the BEFORE catalogue contains the most confusable pair in the chapter. The rule is lexical, and two descriptions too *short* to share vocabulary slip straight past it. `order_status` and `shipment_status` share no content words at all beyond what the stop list removes.

So the linter's most sophisticated rule is blind to precisely the failure the chapter opens with, and the crude rules — length, and "does it say when not to use this" — catch it. Chapter 22 published its own detector's blind spot; this is the same discipline, and it has a design lesson attached: **a similarity check cannot find ambiguity between two things that say nothing.** Emptiness and overlap look identical to a lexical rule and are opposite problems.

Result 5 — why a long description is nearly free

```
catalogue content words: before=11 after=161 (14.64x)
  short descriptions    900 tokens ->  45135.0 billed units / 50 turns
  long  descriptions   2400 tokens ->  54360.0 billed units / 50 turns
the extra 1,500 description tokens cost 150 base-input units per
  cached request
one wrong tool call costs a whole extra round trip: at a 20,000-token
  cached history that is 2000 units of input plus its output
```

The rewrite is **14.64 times** more prose and, over fifty cached turns, costs about twenty per cent more on the input line — because after the first write it is read at 0.1×. Per request the extra 1,500 tokens cost **150 base-input units**. One wrong tool call costs an entire extra round trip: **2,000 units** of cached input plus a full generation, plus the latency, plus whatever the wrong tool did.

13.3 to 1, before counting side effects. A description has to prevent one wrong call in thirteen requests to pay for itself, and Result 1 suggests the real rate is nowhere near that marginal.

What the project establishes

Four numbers and one boundary. **34 of 40** trials on the one-line catalogue decided by a tie. **p = 0.0129** paired, against overlapping unpaired intervals. **Zero** discordant pairs from a plausible-sounding reword. **13.3×** the cost of a wrong call against the whole description budget. And the boundary: none of this measures Claude. It measures whether your catalogue gives Claude anything to work with.

33.18 Hands-on Labs

Lab 1 — Count your ties (success: a tie rate for your real catalogue). Run `affordance.run` over twenty real user requests and report ties, not accuracy.

Lab 2 — Write the reciprocal sentences (success: every confusable pair names the other). Find your two closest tools and add one sentence to each.

Lab 3 — Lint (success: zero errors). Run `lintdesc.lint_catalogue`; fix `TOO_SHORT` and `NO_WHEN_NOT` first.

Lab 4 — Pair a change (success: a discordant-pair count and an exact p). Never compare two catalogues on different request sets.

Lab 5 — Price your descriptions (success: a ratio). Compare the cached cost of your description budget against the cost of one wrong tool call on your history size.

33.19 Lab Solutions

Lab 1. Ties are the honest headline because they do not depend on the scorer being good — only on it being deterministic. A tie rate above about a quarter means the catalogue is under-specified regardless of what a stronger reader would do with it.

Lab 2. Write both directions. A single-sided exclusion moves the ambiguity rather than removing it: the tool that was told to stand down still has no reason to think the other one should step forward.

Lab 3. Fix `NO_WHEN_NOT` before `NO_OMISSIONS`. Both matter, but only the first does work at selection time; the second does work after the tool has been chosen.

Lab 4. Same requests, same order, matched by index. Report `improved`, `regressed` and the exact test. If discordant pairs are zero, you have measured that the change did nothing on this set — which is a result, not a failure.

Lab 5. Use the cached rate (0.1×) for the description and the full round-trip cost for the wrong call, and remember the wrong call also costs output tokens, latency, and — if the tool had a side effect — Chapter 34's problem.

33.20 Best Practices

BEST PRACTICE

1. **Three to four sentences minimum**, more for complex tools.
2. **Say when the tool should not be used, and name the alternative.** Write it in both directions for every confusable pair.
3. **Say what the tool does not return.**
4. **Describe every parameter**, including the optional ones.
5. **Namespace names by service**, and spend the sixty-four characters.
6. **Use readable enum members**; they are constraints the grammar enforces *and* affordances.
7. **Consolidate genuinely adjacent operations** behind an `action` enum, and let the enum carry the distinction.
8. **Return semantic, stable identifiers** — slugs or UUIDs, never internal row ids.
9. **Return only the fields needed for the next step.**
10. **Keep descriptions byte-stable**: no timestamps, versions or per-user detail in the cached prefix.
11. **Treat a description change as a behaviour change**: review it, and run it against a pinned suite.
12. **Pair every comparison** and report an interval; a proportion alone is not a finding.
13. **Publish negative results**, including the reword that did nothing.
14. **Say what your measurement does not establish**, especially when it is a proxy.

33.21 Common Mistakes

COMMON MISTAKE

The one-line description

Symptom: the model picks plausible-but-wrong tools, apparently at random, and prompting the system prompt does not help. **Cause:** the catalogue expresses no preference. Measured here: 34 of 40 trials tie. **Fix:** the documented five items, three to four sentences each. **Detection:** count ties, not accuracy; and lint for `TOO_SHORT`.

COMMON MISTAKE

Two tools that never mention each other

Symptom: requests split unpredictably between an adjacent pair. **Cause:** each description is individually accurate and neither draws the boundary. **Fix:** reciprocal exclusion — one sentence in each naming the other. **Detection:** a confusion matrix over your own requests; the evidence is off the diagonal.

COMMON MISTAKE

Judging a description change by the headline number

Symptom: a change is adopted or abandoned on a 25-point swing with no interval. **Cause:** an unpaired comparison at a sample size that cannot support it — here the Wilson intervals overlap. **Fix:** pair the trials and test the discordant pairs. **Detection:** compute the interval before you write the summary.

COMMON MISTAKE

Believing a proxy measured the model

Symptom: a claim about Claude's selection accuracy sourced from an offline harness. **Cause:** conflating a property of the catalogue with a property of the model. **Fix:** state the boundary in the report, not just in the docstring. **Detection:** ask what would have to be true of the model for the claim to hold; if you cannot say, you do not have the claim.

COMMON MISTAKE

Bloated tool responses

Symptom: context fills quickly, later turns degrade, cache reads climb. **Cause:** returning an entire API payload rather than what the next step needs. **Fix:** project to high-signal fields; return slugs, not internal ids. **Detection:** measure tokens per `tool_result` and compare with what the next turn actually used.

COMMON MISTAKE

Unstable descriptions

Symptom: cache reads at zero after a "documentation improvement". **Cause:** a version string, a date or a per-user detail interpolated into a description. **Fix:** static strings; move anything dynamic after the cache breakpoint. **Detection:** Chapter 32's hash-the-array check.

COMMON MISTAKE

Tuning descriptions on the set you measure on

Symptom: a large improvement that does not reproduce in production. **Cause:** the winner's curse — Chapter 24 measured the best of twenty identical prompts falling 3.86 points on fresh data. **Fix:** hold out requests; confirm the winner separately. **Detection:** report how many variants you tried before choosing.

33.22 Performance Considerations

Descriptions are prefill, and prefill is what caching removes. The measured trade is twenty per cent more on the input line over fifty cached turns for **14.64×** the prose, which is the whole economic argument in one sentence. Chapter 11's rule is unchanged: decode dominates latency, and a longer description does not slow generation.

Two second-order effects are worth watching. Longer descriptions make the `tools` array larger, and a larger array crossing a model's minimum cacheable length is a benefit (Chapter 30) while a larger array rebuilt per request is a bigger loss. And under **tool search** with `defer_loading`, descriptions of deferred tools are not in the prefix at all — they arrive as `tool_reference` blocks in the message history when discovered, which changes both the cost profile and, per Chapter 31, the block accounting that the 20-block lookback window counts.

The grammar cache is indifferent to all of this: Chapter 23 established that it is keyed on schema structure and the tool set, and *not* on names or descriptions. Editing prose costs the prompt cache and nothing else.

33.23 Security Considerations

A description is an instruction in the highest-authority channel available to a tool. It is assembled into the tool-use system prompt, above your own system prompt in the constructed prefix. Anything that reaches it reaches operator-adjacent authority — so a description must never be assembled from user input, tenant configuration or anything fetched at runtime. Chapter 21 established the authority boundary; this is the least obvious place it leaks.

Descriptions and enum values are shipped on every request and appear in logs. Chapter 32 recorded that the schema is cached separately and does not receive the same PHI protections; the same reasoning covers internal hostnames, table names and customer identifiers appearing in an enum.

Response shaping is a security control as well as an economy. A tool that returns whole records exports more than the model needs, into a context that Chapter 58 will show is attackable. Returning three fields instead of forty is least privilege applied to data.

"When not to use this tool" is not an authorisation control. It steers a cooperative model and does nothing against a redirected one. Chapter 34 builds the control that actually holds: the call site authorises the caller, never the request.

33.24 Real Production Examples

A support desk. The chapter's catalogue. The single highest-value change was the reciprocal sentence between order status and shipment tracking; the second was giving every parameter a description.

A large internal library. Two hundred tools behind `defer_loading` and tool search. Namespacing stopped being a style rule and became the retrieval key: `finance_`, `hr_`, `infra_` prefixes are what make a search over two hundred documents tractable.

A coding agent. Anthropic-schema tools where they exist — Chapter 31's trained-in signatures — and, for the rest, descriptions that say what the tool does *not* do, because in a codebase almost every tool is adjacent to two others.

A regulated workflow. Descriptions reviewed like code, pinned in a Chapter 24 regression suite, and a CI check that fails on any description edit without a corresponding suite run.

33.25 Certification Notes

Concept	Exam domain	Weight
Descriptions as the primary lever on tool performance	D2 Tool Design & MCP Integration	18%
The five documented description items	D2 Tool Design & MCP Integration	18%
Namespacing and tool search	D2 Tool Design & MCP Integration	18%
Consolidation and selection ambiguity	D1 Agentic Architecture & Orchestration	27%
Response shaping and context economy	D5 Context Management & Reliability	15%
Measuring a change with pairing and intervals	D4 Prompt Engineering & Structured Output	20%
Descriptions in the cached prefix	D5 Context Management & Reliability	15%

CERTIFICATION NOTE

Exam Tips

- The description is *"by far the most important factor in tool performance"* — quoted, not paraphrased.
- **Three to four sentences minimum.**
- The five items include **when NOT to use it** and **what it does not return**.
- **Namespace by service**, *"especially important when using tool search"*.
- Consolidation's documented benefit is **reduced selection ambiguity**, not only budget.
- Responses should return **semantic, stable identifiers** and only next-step fields.
- Descriptions live in the **cached prefix** and are read at **0.1×** — length is nearly free, instability is not.
- The **grammar cache is not invalidated** by name or description changes; the **prompt cache is**.
- A proportion without an interval is not a finding, and **overlapping intervals do not disprove a paired difference**.

33.26 Twenty Practice Questions

1. Quote the documented claim about descriptions and tool performance.
2. What is the documented minimum description length?
3. Name the five things a description should cover.
4. Which two are most often omitted, and why do they matter most?
5. What is a tie, and why report it instead of accuracy?
6. Why does namespacing matter *"especially"* under tool search?
7. Give the documented reason to consolidate related operations.
8. What kind of identifier should a tool return, and why?

9. Two catalogues score 12/40 and 22/40 with overlapping Wilson intervals. Is the difference established?
10. Same data, 12 improvements and 2 regressions among 14 discordant pairs. Now?
11. Why are the 26 concordant trials excluded from the test?
12. How many cases for a ±0.05 interval around 0.55?
13. What does an unpaired comparison cost you, per Chapter 24?
14. Which cache does a description edit invalidate, and which does it not?
15. What does the extra prose cost per cached request, and what does one wrong tool call cost?
16. Why can a lexical overlap rule miss the most confusable pair?
17. What is the whole surface the model sees for a tool?
18. Where must a description never be assembled from?
19. What does "when not to use this tool" not protect against?
20. Your harness says accuracy rose 25 points. What must you say alongside it?

33.27 Answers

1. *"Provide extremely detailed descriptions. This is by far the most important factor in tool performance."*
2. *"Aim for at least 3–4 sentences for each tool description, more if the tool is complex."*
3. What it does; when it should be used and when it shouldn't; what each parameter means; caveats and limitations, *"such as what information the tool does not return"*; at the documented length.
4. **When not to use it** and **what it does not return.** They are the only two that do work at *selection* time, when the model is choosing between tools rather than reading about one.
5. A tie is a trial where the top two tools score identically — an **absent** preference resolved by something arbitrary. It is reported because it does not depend on the scorer being good, only deterministic. Measured: **34 of 40** on the one-line catalogue.
6. Under `defer_loading`, tools are discovered by search rather than read from a list, so the name is the retrieval key. A name that does not say which service it belongs to is a document with no title.
7. *"Fewer, more capable tools reduce selection ambiguity and make your tool surface easier for Claude to navigate."*
8. *"Semantic, stable identifiers (for example, slugs or UUIDs) rather than opaque internal references"* — readable, reasoning-friendly, and the same across environments.
9. **No.** [0.181, 0.454] and [0.398, 0.693] overlap; the unpaired comparison establishes nothing at n = 40.
10. **Yes.** An exact two-sided test on the discordant pairs gives **p = 0.0129**.
11. Because they carry no information about the difference — both catalogues got them right, or both wrong. All the evidence is where the two systems disagree.
12. **410.**
13. Roughly **3.51×** the sample size, per Chapter 24's measurement (903 per arm unpaired against 515 paired).

14. It invalidates the **prompt cache** (any byte change) and **not** the **grammar cache**, which is keyed on schema structure and the tool set.
15. **150 base-input units** per cached request for the extra 1,500 tokens; **2,000 units** of cached input plus a full generation for one wrong call — about **13.3×**.
16. Because it is lexical: two descriptions too **short** to share vocabulary look no more similar than two unrelated ones. Emptiness and overlap are opposite problems that a similarity rule cannot tell apart.
17. Tool name, description, parameter names, parameter descriptions, enum values — and `input_examples` if present. Nothing else; *"Claude never sees your implementation."*
18. From user input, tenant configuration or anything fetched at runtime. It is assembled into the tool-use system prompt, at operator-adjacent authority.
19. A **redirected** model. It steers a cooperative one. The control that holds is call-site authorisation of the caller — Chapter 34.
20. The interval, the sample size, whether the comparison was paired, how many variants you tried, and — if it is a proxy — what it does not establish.

33.28 Ten Practical Exercises

1. Compute the tie rate for your real catalogue on twenty real requests.
2. Add reciprocal exclusion sentences to your closest pair and re-measure, paired.
3. Lint every description and fix `TOO_SHORT` and `NO_WHEN_NOT`.
4. Add a description to every parameter, including optional ones.
5. Rename every tool with a service prefix and measure the token cost.
6. Replace every internal id in a tool response with a slug.
7. Cut one tool's response to the fields the next turn actually used.
8. Grep descriptions for interpolated values and remove them.
9. Pin your tool catalogue in a Chapter 24 regression suite.
10. Compute the sample size you would need to estimate your selection accuracy to ±5 points.

33.29 Five Challenge Labs

1. **The reciprocal-exclusion generator.** From a confusion matrix, propose the sentence pairs that would separate each confusable pair, and measure the effect paired.
2. **The stronger reader.** Replace the bag-of-words scorer with an embedding model (Chapter 6) and check whether the tie count and the ranking of catalogues agree; report where they diverge.
3. **The held-out harness.** Split requests into development and confirmation sets and quantify your own winner's curse across eight description variants.
4. **The response slimmer.** Instrument which returned fields are referenced in the following turn and propose a projection.
5. **The catalogue budget.** Combine this chapter's linter with Chapter 32's budget and Chapter 30's cache planner into one gate that fails a build on prose, schema or cache grounds.

33.30 Ten Interview Questions

1. Why is the description the most important field in a tool definition?
2. What are the two sentences almost every description is missing?
3. How would you diagnose two tools competing for the same requests?
4. What is a tie, and why is it a better headline than accuracy?
5. Why does a longer description not cost what people expect?
6. How would you measure whether a description change helped?
7. Your intervals overlap but the paired test is significant. Which do you report?
8. Why does namespacing matter more as a library grows?
9. What should a tool return, and what should it not?
10. What can an offline harness establish about tool selection, and what can it not?

33.31 Summary

The most consequential field in a tool definition is the one with no type, no compiler and no test: the description. The documentation calls it *"by far the most important factor in tool performance"*, and three facts established earlier in this book explain why that should be true rather than merely asserted. Claude never sees your implementation, so the description is the entire interface. The description is assembled into the tool-use system prompt, so it is a prompt and every rule from Chapter 20 governs it. And it lives in the cached prefix, so after the first request it is read at a tenth of the input price, which makes brevity not a saving but a bet.

The documented list has five items and the two that get dropped are the two that do work at selection time: **when the tool should not be used**, and **what it does not return**. Everything else in a description helps a model that has already chosen; those two help it choose. The cheapest fix in the chapter is reciprocal exclusion — for every pair of tools that could answer the same request, one sentence in each naming the other and saying which questions belong to it. It has to be written twice, because a single-sided prohibition moves the ambiguity instead of removing it.

The measurement is where the chapter earns its keep, and it earns it twice. On the one-line catalogue, **thirty-four of forty** requests end in a tie: the top two tools score identically and something arbitrary finishes the job. That is the true shape of a bad catalogue — not wrong preferences but absent ones — and it means the confusion matrix must be read alongside the tie count or it tells a false story. After the rewrite, ties fall to sixteen, the mean margin rises from 0.208 to 1.011, and proxy accuracy goes from 12/40 to 22/40. But the two Wilson intervals overlap, so the unpaired comparison establishes nothing at forty cases; pairing the trials isolates fourteen discordant pairs split twelve to two, and an exact test returns **p = 0.0129**. Both readings are correct simultaneously, and reporting only one of them would be either overclaiming or abandoning a real improvement. The chapter also publishes a reword that sounds like an improvement, produces **zero** discordant pairs, and is therefore not one — and a linter whose most sophisticated rule is blind to the very pair the chapter opens with, because a lexical similarity check cannot distinguish two descriptions that overlap from two that say nothing at all.

And the boundary, restated because it is the condition on everything above: the harness does not measure Claude. It measures whether a catalogue gives Claude anything to work with, which is a fact

about your strings rather than a guess about a model. That is a narrower claim than "better descriptions improve tool selection", and it is one that holds. Chapter 34 turns from what a tool says about itself to what it owes its caller when things go wrong — the error contract, validation at the call site that Chapter 32 deferred five times, and the idempotency that Chapter 29 showed the protocol will not provide.

KEY TAKEAWAYS

1. **The description is the entire interface**, and the documentation calls it *"by far the most important factor in tool performance"*.
2. **Three to four sentences minimum**, covering what it does, when to use it, **when not to**, what each parameter means, and **what it does not return**.
3. **Reciprocal exclusion:** every confusable pair names the other, in both directions.
4. **Count ties, not accuracy.** 34 of 40 trials on the one-line catalogue were decided by nothing at all.
5. **A confusion matrix without a tie count tells a false story** — the apparent winner was collecting ties, not preferences.
6. **Overlapping intervals did not disprove the improvement; pairing established it** at p = 0.0129, and 410 cases would be needed to estimate the level to ±5 points.
7. **Publish the reword that changed nothing**, and the linter rule that missed the pair it was written for.
8. **Namespace by service** — it is the retrieval key under tool search, not a style preference.
9. **Shape the response:** semantic stable identifiers, only next-step fields, because bloat costs tokens, blocks and attention.
10. **A long description costs 150 units per cached request; a wrong call costs 2,000** — about **13.3×**.

33.32 Glossary

Term	Definition
Affordance	Anything the model can see about a tool: name, description, parameter names and descriptions, enum values, examples
Concordant pair	A trial two catalogues answer the same way; carries no information about a difference
Confusion matrix	Expected tool against chosen tool; the evidence is off the diagonal
Discordant pair	A trial the two catalogues answer differently; the only trials a paired test uses
Margin	The score gap between the chosen tool and the runner-up
Proxy measurement	A transparent stand-in for a quantity you cannot measure directly; here, a lower bound on catalogue distinguishability
Reciprocal exclusion	Two adjacent tools each naming the other and saying which requests belong to it
Response shaping	Returning only high-signal, next-step fields with semantic stable identifiers
Selection ambiguity	Two or more tools being plausible answers to the same request
Surface	The concatenation of everything the model is given about a tool

Term	Definition
Tie	A trial where the top two tools score identically; an absent answer, not a wrong one
Tool search	Discovering deferred tools at run time; makes the name a retrieval key
Winner's curse	The upward bias in the best of k variants, even when all k are identical

33.33 Further Reading

Primary sources, verified 2 August 2026 at `platform.claude.com/docs/en`:

- `agents-and-tools/tool-use/define-tools` — the best-practices section: extremely detailed descriptions, the 3–4 sentence floor, consolidation, namespacing, response shaping, and the good-versus-poor description contrast.
- `agents-and-tools/tool-use/tool-search-tool` and `agents-and-tools/tool-use/manage-tool-context` — discovery and context economy for large libraries.
- `agents-and-tools/tool-use/tool-use-with-prompt-caching` — `defer_loading` and breakpoint placement, verified in Chapter 30.
- Anthropic Engineering, *Writing tools for agents*, `https://www.anthropic.com/engineering/writing-tools-for-agents` — the essay the documentation points to for consolidation, naming and response shaping.
- `test-and-evaluate/develop-tests` — the evaluation discipline this chapter borrows from Chapter 24.

Where to go next in this book. Chapter 34 builds the error contract, call-site validation and idempotency. **Chapter 35** takes a catalogue up to orchestration across many turns. **Chapter 36** opens Part 7 with MCP, where the descriptions are written by someone else. **Chapter 53** covers retrieval, which is what tool search is doing to your library. **Chapter 57** builds the observability that turns a confusion matrix into a production dashboard.